FRONT END DEVELOPER - REAL TIME STOCK SCREENER

Category: Front-End Development | Financial Technology Engineering

Timeline: 15 Days | AI-Accelerated Innovation & Research Project

Target: Undergraduate / MBA / Engineering Students

TASK:

Design, build, and deliver a production-grade real-time stock screener application - a direct competitor to Screener.in and Finviz - as Lead Front-End Developer at a fictional retail brokerage firm. The application must handle a universe of 5,000+ stock records with sub-200-millisecond filter response times, render interactive candlestick charts with a minimum of five technical overlay indicators, implement virtual scrolling for seamless large-dataset navigation, and stream live price updates through a WebSocket simulation layer - all built on React 18 and Next.js 14 with rigorous TypeScript discipline, Zustand-based state management, and TanStack Table virtualisation. The solution must demonstrate production-grade front-end engineering across six domains: React component architecture using compound component and custom hook patterns, high-performance data grid rendering, financial charting library integration, WebSocket real-time data handling, advanced filter engine design, and performance optimization validated through documented benchmark results.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

PART E: NO-CODE TOOLS & PLATFORMS GUIDE

PART F: SUBMISSION PROTOCOL

Important Note:

This Project is designed, developed, and administered solely by Zetheta Algorithms Private Limited ("Zetheta") for the purpose of assessing candidates, students, or event participants on behalf of engaging employers, institutes, or event organisers. Completion of this Project does not guarantee employment, academic outcomes, awards, or any other result - all such decisions remain at the sole discretion of the respective engaging party. Zetheta may use AI-assisted tools to evaluate submissions; such outputs are indicative in nature and may be inaccurate or incomplete. By participating, you consent to your performance data, scores, and submissions being shared with relevant engaging parties for assessment and decision-making purposes.

EXECUTIVE SUMMARY

This project positions students as Lead Front-End Developer at a fictional retail brokerage firm, requiring them to master the convergence of high-performance React engineering, financial data visualisation, and real-time WebSocket architecture. Unlike typical front-end projects that demonstrate basic CRUD interfaces or static component libraries, this project demands simultaneous fluency in virtual DOM optimisation, large-dataset rendering strategy, and financial charting library integration on one side, and production engineering practices including TypeScript strict mode, error boundary architecture, custom hook design, and performance benchmarking on the other.

The core challenge students must solve: How can a browser-based stock screening application handle a live universe of 5,000+ instruments - each with real-time price updates, multi-dimensional filter logic, sortable column state, and interactive candlestick charting - without frame drops, filter lag, or memory leaks, and without the codebase collapsing under its own complexity as feature requirements scale? The answer lies in a layered performance architecture built across four engineering disciplines - virtual scrolling to eliminate DOM bloat, memoised filter engines to decouple computation from render cycles, WebSocket delta patching to minimise state churn, and React Suspense-based code splitting to keep initial bundle weight under control - where every architectural decision is benchmarked rather than assumed, and every component boundary is drawn to isolate failure, not just organise files.

Students will engineer and validate a complete stock screener across six progressive technical workstreams: a virtualised data grid handling 5,000+ rows with sub-200ms filter response and real-time cell-level price updates; a financial charting module with candlestick rendering and at least five technical overlay indicators including moving averages, RSI, and Bollinger Bands; a compound-component filter panel supporting numeric range, dropdown, and boolean filter types with shared Zustand state; a WebSocket simulation layer delivering streaming price updates with reconnection logic and connection state UI; a comprehensive testing suite covering unit, integration, and performance benchmarks with documented pass/fail thresholds; and a production deployment on Vercel with Lighthouse performance scores, Core Web Vitals benchmarks, and a professional engineering README. Every component must demonstrate a documented architectural rationale, every performance claim must be supported by a measured benchmark, and every hook must be independently testable outside the UI layer.

The global fintech engineering talent market is experiencing acute demand for front-end developers who can operate at the intersection of financial data complexity and browser performance engineering - a pairing that most developers encounter only years into their careers, and that most university curricula never address. The gap between a developer who can build React interfaces and one who can architect a real-time financial data platform that performs correctly under production load remains one of the most commercially valuable skill differentials in front-end engineering. This project closes it directly, equipping students with the React architecture depth, performance optimisation methodology, and financial UI domain knowledge that mid-to-senior front-end engineering roles at fintech firms, trading platforms, and retail brokerage products demand.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART A: TRAINING MATERIAL & LEARNING RESOURCES

SECTION A1: REACT AND NEXT.JS ARCHITECTURE FUNDAMENTALS

The application must be built using React 18 or later with Next.js 14 or later as the framework. Understanding the following concepts is mandatory before writing any production code.

A1.1 Component Architecture Patterns

A well-architected stock screener requires a clear separation between presentational components, container components, and hook-based logic. The following patterns must be demonstrated in the submission:

- **Compound Component Pattern:** Used for the filter panel where multiple filter controls share state and communicate through a shared context. Each filter type (numeric range, dropdown, boolean toggle) is a child component that registers itself with the parent FilterPanel.
- **Render Props and Custom Hooks:** Data fetching and caching logic must be extracted into custom hooks (useStockData, useWebSocket, useFilterEngine) that can be tested independently of the UI layer.
- **Controlled vs Uncontrolled Components:** All filter inputs must be controlled components with state managed through Zustand or Redux. Uncontrolled form elements are not acceptable for a financial screening application where deterministic state is critical.
- **Error Boundary Implementation:** Each major feature area (chart, data grid, filter panel) must have its own error boundary with graceful fallback UI and error reporting to a logging service.
- **Suspense Integration:** React Suspense must be used for code-splitting chart components and lazy-loading heavy dependencies like the charting library.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

A1.2 Next.js App Router and Server Components

The project must use the Next.js App Router with a clear understanding of when to use Server Components versus Client Components:

- Server Components: Initial page shell, metadata, static filter configuration loading, SEO-friendly stock detail pages.
- Client Components: The entire screener interface (filter panel, data grid, charts) must be client components because they require interactivity, WebSocket connections, and real-time state updates.
- Route Groups: Use route groups to organise the screener, watchlist, and stock detail pages without affecting the URL structure.

- Parallel Routes: Consider parallel routes for the stock detail modal that overlays the screener grid without losing scroll position.

A1.3 Code Example: Custom Hook for Stock Data

The following pattern demonstrates the expected level of hook architecture:

```
// hooks/useStockScreener.ts
import { useQuery, useQueryClient } from
'@tanstack/react-query';
import { useStockStore } from '@/stores/stockStore';
import { filterStocks } from '@/lib/filterEngine';

interface UseStockScreenerOptions {
  enabled?: boolean;
  refetchInterval?: number;
}

export function useStockScreener(options:
UseStockScreenerOptions = {}) {
  const { enabled = true, refetchInterval = 30000 } = options;
  const filters = useStockStore((state) =>
state.activeFilters);
  const sortConfig = useStockStore((state) =>
state.sortConfig);
  const queryClient = useQueryClient();

  const { data: allStocks, isLoading, error } = useQuery({
    queryKey: ['stocks', 'universe'],
    queryFn: () => fetch('/api/stocks').then(r => r.json()),
    staleTime: 5 * 60 * 1000,
    enabled,
  });

  const filteredStocks = useMemo(() => {
    if (!allStocks) return [];
    return filterStocks(allStocks, filters, sortConfig);
  }, [allStocks, filters, sortConfig]);

  const prefetchStockDetail = useCallback((symbol: string) => {
    queryClient.prefetchQuery({
      queryKey: ['stock', symbol, 'detail'],
      queryFn: () => fetch(`/api/stocks/${symbol}`).then(r =>
r.json()),
    });
  }, [queryClient]);

  return { stocks: filteredStocks, isLoading, error,
prefetchStockDetail };
}
```

SECTION A2: DATA GRID PERFORMANCE ENGINEERING

The data grid is the centrepiece of the stock screener. It must display 5,000 or more rows with smooth scrolling, instant sorting, and real-time cell updates. This section covers the engineering principles required to achieve production-grade performance.

A2.1 Virtual Scrolling Architecture

Virtual scrolling (also called windowed rendering) is the technique of rendering only the visible rows plus a small overscan buffer, rather than mounting all 5,000 or more rows in the DOM. The intern must implement this using TanStack Table with TanStack Virtual or a comparable virtualisation library.

- **Row Height Strategy:** Fixed row heights (36 pixels recommended) enable O(1) scroll position calculation. Variable row heights require a height cache and are not recommended for this project.
- **Overscan Count:** An overscan of 10 to 15 rows above and below the viewport prevents blank flashes during fast scrolling. The intern must benchmark different overscan values and document the trade-off between smoothness and memory usage.
- **Scroll Event Throttling:** Use requestAnimationFrame-based throttling for scroll events. Never use setTimeout or setInterval for scroll handling.
- **DOM Recycling:** TanStack Virtual reuses DOM nodes as the user scrolls. The intern must ensure that cell renderers are pure functions that do not retain stale closures or event listeners.

A2.2 Column Configuration and Cell Rendering

Each column in the data grid must have explicit configuration covering data type, sort comparator, filter type, cell renderer, and width:

```
// columns/stockColumns.tsx
import { createColumnHelper } from '@tanstack/react-table';
import { PriceCell } from '@/components/cells/PriceCell';
import { ChangeCell } from '@/components/cells/ChangeCell';
import { VolumeCell } from '@/components/cells/VolumeCell';

const columnHelper = createColumnHelper<Stock>();

export const stockColumns = [
  columnHelper.accessor('symbol', {
    header: 'Symbol',
    size: 100,
    cell: (info) => (
      <span className="font-mono font-semibold text-blue-700">
        {info.getValue()}
      </span>
    ),
    enableSorting: true,
    sortingFn: 'alphanumeric',
  }),
```

```
  columnHelper.accessor('lastPrice', {
    header: 'LTP',
    size: 120,
    cell: (info) => <PriceCell value={info.getValue()} />,
    sortingFn: 'basic',
    meta: { filterType: 'range', unit: 'INR' },
  }),
  columnHelper.accessor('changePercent', {
    header: '% Change',
    size: 100,
    cell: (info) => <ChangeCell value={info.getValue()} />,
    sortingFn: 'basic',
  }),
  columnHelper.accessor('volume', {
    header: 'Volume',
    size: 120,
    cell: (info) => <VolumeCell value={info.getValue()} />,
    sortingFn: 'basic',
    meta: { filterType: 'range' },
  }),
  // ... 30+ additional columns
];
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

A2.3 Performance Benchmarks Required

The following benchmarks are mandatory. The intern must include a performance audit report (using Lighthouse and React DevTools Profiler) with their submission:

Metric	Target	Measurement Method	Weight
Initial Load (LCP)	< 2.5 seconds	Lighthouse / Chrome DevTools	Critical
Filter Response Time	< 200ms for 5,000 rows	Performance.now() instrumentation	Critical
Sort Response Time	< 150ms for 5,000 rows	Performance.now() instrumentation	Critical
Scroll FPS	> 55 FPS during fast scroll	Chrome DevTools Performance tab	High
Memory Usage	< 150MB with 5,000 rows	Chrome Task Manager	High
WebSocket Update Latency	< 50ms from receipt to render	Custom instrumentation	Medium
Time to Interactive (TTI)	< 3.5 seconds	Lighthouse	Medium
Cumulative Layout Shift	< 0.1	Lighthouse / Web Vitals	Medium

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A3: FINANCIAL CHARTING LIBRARY INTEGRATION

The stock screener must include an interactive candlestick chart for any selected stock. The chart must support at least five technical overlay indicators and respond to real-time price updates from the WebSocket simulation.

A3.1 Recommended Charting Libraries

The intern may choose one of the following charting libraries. Each has distinct trade-offs:

Library	Pros	Cons	Recommendation
Lightweight Charts (TradingView)	Purpose-built for financial data, excellent performance, WebSocket-friendly API, small bundle size (~40KB)	Limited customisation for non-financial charts, no built-in indicator library	PRIMARY RECOMMENDATION
Recharts	React-native, declarative API, good for simple charts	Not optimised for financial data, no native candlestick support, poor performance with large datasets	ACCEPTABLE for supplementary charts only
D3.js (custom)	Maximum flexibility, full control over rendering	Significant development time, requires deep SVG/Canvas knowledge, no React integration out of box	ADVANCED OPTION for experienced developers
Apache ECharts	Comprehensive chart types, good performance, built-in financial charts	Large bundle size (~800KB), imperative API feels non-React	ACCEPTABLE alternative to Lightweight Charts

A3.2 Required Technical Indicators

The chart component must support the following five mandatory overlay indicators. The intern must implement the calculation logic from scratch (no pre-built indicator libraries) to demonstrate understanding of the mathematics:

1. **Simple Moving Average (SMA):** Calculate the arithmetic mean of closing prices over a configurable period (default: 20, 50, 200 days). Display as a smooth line overlay on the candlestick chart. Formula: SMA(n) = (1/n) * sum of closing prices over n periods.

2. **Exponential Moving Average (EMA):** Weighted moving average with exponential decay. Multiplier = 2 / (period + 1). The EMA gives more weight to recent prices. Required periods: 12 and 26 (for MACD calculation).

3. **Bollinger Bands:** Upper Band = SMA(20) + 2 * standard deviation. Lower Band = SMA(20) - 2 * standard deviation. Middle Band = SMA(20). The bands must dynamically adjust as new data arrives via WebSocket.

4. **Relative Strength Index (RSI):** RSI = 100 - (100 / (1 + RS)) where RS = average gain over n periods / average loss over n periods. Default period: 14. Display in a separate sub-chart below the main candlestick chart with overbought (70) and oversold (30) reference lines.

5. **Volume Profile:** Horizontal histogram showing volume distribution at each price level. Aggregate volume into price buckets and display as a horizontal bar overlay on the chart. This indicator is essential for identifying support and resistance zones.

A3.3 Chart Component Architecture

```
// components/StockChart/StockChart.tsx
import { useRef, useEffect, useCallback } from 'react';
import { createChart, IChartApi, ISeriesApi } from
'lightweight-charts';
import { useWebSocket } from '@/hooks/useWebSocket';
import { calculateSMA, calculateEMA, calculateBollinger,
         calculateRSI, calculateVolumeProfile } from
'@/lib/indicators';

interface StockChartProps {
  symbol: string;
  timeframe: '1D' | '1W' | '1M' | '3M' | '1Y' | '5Y';
  indicators: IndicatorConfig[];
  height?: number;
}

export function StockChart({ symbol, timeframe, indicators,
height = 500 }: StockChartProps) {
  const chartContainerRef = useRef<HTMLDivElement>(null);
  const chartRef = useRef<IChartApi | null>(null);
  const candlestickRef = useRef<ISeriesApi<'Candlestick'> |
null>(null);
  const indicatorSeriesRef = useRef<Map<string,
ISeriesApi<any>>>(new Map());

  // Initialise chart on mount
  useEffect(() => {
    if (!chartContainerRef.current) return;
    const chart = createChart(chartContainerRef.current, {
      width: chartContainerRef.current.clientWidth,
      height,
      layout: { textColor: '#1a1a1a', background: { color:
'#ffffff' } },
```

```
      grid: { vertLines: { color: '#f0f0f0' }, horzLines: {
color: '#f0f0f0' } },
      crosshair: { mode: 0 },
      timeScale: { timeVisible: true, secondsVisible: false },
    });
    chartRef.current = chart;
    candlestickRef.current = chart.addCandlestickSeries({
      upColor: '#22c55e', downColor: '#ef4444',
      borderUpColor: '#16a34a', borderDownColor: '#dc2626',
    });
    return () => { chart.remove(); chartRef.current = null; };
  }, [height]);

  // Subscribe to real-time updates
  useWebSocket(`price:${symbol}`, useCallback((data) => {
    candlestickRef.current?.update(data.candle);
    // Recalculate and update indicator overlays
    updateIndicators(data.candle);
  }, [symbol]));

  return <div ref={chartContainerRef} className="w-full
rounded-lg border" />;
}
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A4: WEBSOCKET SIMULATION FOR REAL-TIME DATA

Since the project does not connect to a live market data provider, the intern must build a WebSocket simulation layer that mimics realistic market behaviour. This section defines the simulation requirements and architecture.

A4.1 WebSocket Server Simulation

The simulation server must be implemented as a Next.js API route or a standalone Node.js process. It must generate realistic price movements using a geometric Brownian motion model:

```
// lib/priceSimulator.ts
export function simulateNextPrice(
  currentPrice: number,
  volatility: number = 0.02,
  drift: number = 0.0001,
  dt: number = 1 / 252  // 1 trading day
): number {
  const randomShock = Math.sqrt(dt) * gaussianRandom();
  const priceChange = drift * dt + volatility * randomShock;
  return currentPrice * (1 + priceChange);
}

function gaussianRandom(): number {
  // Box-Muller transform for normal distribution
  const u1 = Math.random();
  const u2 = Math.random();
  return Math.sqrt(-2 * Math.log(u1)) * Math.cos(2 * Math.PI *
u2);
}

// Simulate correlated sector movements
export function simulateSectorMovement(
  stocks: Stock[],
  sectorCorrelation: number = 0.6
): Map<string, number> {
  const sectorShock = gaussianRandom();
  const updates = new Map<string, number>();
  for (const stock of stocks) {
    const idiosyncratic = gaussianRandom();
    const combinedShock = sectorCorrelation * sectorShock +
      Math.sqrt(1 - sectorCorrelation ** 2) * idiosyncratic;
    const newPrice = simulateNextPrice(stock.lastPrice,
stock.volatility);
    updates.set(stock.symbol, newPrice);
  }
  return updates;
}
```

A4.2 WebSocket Client Hook

The client-side WebSocket hook must handle connection lifecycle, automatic reconnection with exponential backoff, and efficient state batching:

```
// hooks/useWebSocket.ts
import { useEffect, useRef, useCallback } from 'react';
import { useStockStore } from '@/stores/stockStore';

const RECONNECT_DELAYS = [1000, 2000, 4000, 8000, 16000]; //
Exponential backoff

export function useRealtimeUpdates() {
  const wsRef = useRef<WebSocket | null>(null);
  const reconnectAttempt = useRef(0);
        const batchUpdate = useStockStore((s) =>
s.batchUpdatePrices);
   const pendingUpdates = useRef<Map<string, PriceUpdate>>(new
Map());
  const rafId = useRef<number | null>(null);

  const flushUpdates = useCallback(() => {
    if (pendingUpdates.current.size > 0) {
      batchUpdate(pendingUpdates.current);
      pendingUpdates.current = new Map();
    }
    rafId.current = null;
  }, [batchUpdate]);

  const connect = useCallback(() => {
     const ws = new WebSocket(process.env.NEXT_PUBLIC_WS_URL ||
'ws://localhost:3001');
    ws.onmessage = (event) => {
      const data: PriceUpdate = JSON.parse(event.data);
      pendingUpdates.current.set(data.symbol, data);
      if (!rafId.current) {
        rafId.current = requestAnimationFrame(flushUpdates);
      }
    };
    ws.onclose = () => {
      const delay = RECONNECT_DELAYS[
                            Math.min(reconnectAttempt.current,
RECONNECT_DELAYS.length - 1)
      ];
      reconnectAttempt.current++;
      setTimeout(connect, delay);
    };
    ws.onopen = () => { reconnectAttempt.current = 0; };
    wsRef.current = ws;
  }, [flushUpdates]);

       useEffect(()    =>    {    connect();    return    ()    =>
wsRef.current?.close(); }, [connect]);
}
```

A4.3 Real-Time Cell Update Strategy

When a price update arrives via WebSocket, only the affected cells in the data grid should re-render. This requires careful memoisation:

- **Cell-Level Memoisation:** Each cell component must be wrapped in React.memo with a custom comparison function that checks only the cell's data value, not the entire row object.
- **Flash Animation:** Price cells must flash green for price increases and red for price decreases. The flash animation must last exactly 300 milliseconds and must not trigger a re-render of adjacent cells.
- **Batch State Updates:** Multiple price updates arriving within the same animation frame must be batched into a single state update using requestAnimationFrame.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A5: STATE MANAGEMENT ARCHITECTURE

The application state is divided into three categories, each requiring a different management strategy:

State Category	Examples	Management Tool	Rationale
Server State	Stock universe, historical OHLCV data, fundamentals	React Query (TanStack Query)	Automatic caching, background refetching, stale-while-revalidate strategy
Client State	Active filters, sort configuration, selected stock, UI preferences	Zustand	Lightweight, no boilerplate, supports selectors for granular subscriptions
Real-Time State	Live prices, WebSocket connection status, pending updates	Zustand with immer middleware	Mutable draft state for efficient batch updates, compatible with requestAnimationFrame batching

A5.1 Zustand Store Architecture

```
// stores/stockStore.ts
import { create } from 'zustand';
import { immer } from 'zustand/middleware/immer';
import { devtools, persist } from 'zustand/middleware';

interface StockStore {
  // Filter state
  activeFilters: FilterConfig[];
  addFilter: (filter: FilterConfig) => void;
  removeFilter: (filterId: string) => void;
  clearAllFilters: () => void;

  // Sort state
  sortConfig: SortConfig;
  setSortConfig: (config: SortConfig) => void;

  // Selection state
  selectedSymbol: string | null;
  setSelectedSymbol: (symbol: string | null) => void;

  // Real-time prices
  livePrices: Map<string, PriceData>;
```

```
    batchUpdatePrices: (updates: Map<string, PriceUpdate>) =>
void;

  // Watchlist
  watchlist: Set<string>;
  toggleWatchlist: (symbol: string) => void;
}

export const useStockStore =
create<StockStore>()(immer(devtools(persist((set) => ({
  activeFilters: [],
        addFilter: (filter) => set((state) => {
state.activeFilters.push(filter); }),
  removeFilter: (filterId) => set((state) => {
     state.activeFilters = state.activeFilters.filter(f => f.id
!== filterId);
  }),
  clearAllFilters: () => set((state) => { state.activeFilters =
[]; }),
  sortConfig: { column: 'marketCap', direction: 'desc' },
   setSortConfig: (config) => set((state) => { state.sortConfig
= config; }),
  selectedSymbol: null,
      setSelectedSymbol: (symbol) => set((state) => {
state.selectedSymbol = symbol; }),
  livePrices: new Map(),
  batchUpdatePrices: (updates) => set((state) => {
    for (const [symbol, update] of updates) {
          state.livePrices.set(symbol, { ...update, updatedAt:
Date.now() });
    }
  }),
  watchlist: new Set(),
  toggleWatchlist: (symbol) => set((state) => {
                          if (state.watchlist.has(symbol))
state.watchlist.delete(symbol);
    else state.watchlist.add(symbol);
  }),
}), { name: 'stock-store', partialize: (s) => ({ watchlist:
s.watchlist }) })));
```

SECTION A6: ADVANCED FILTER ENGINE DESIGN

The filter engine is the computational core of the screener. It must support 30 or more filter criteria with combinatorial AND/OR logic and execute in under 200 milliseconds for 5,000 records.

A6.1 Required Filter Criteria (Minimum 30)

The following table lists the mandatory filter criteria. The intern may add additional criteria for bonus points:

Category	Filter Name	Type	Range / Options
Fundamentals	Market Capitalisation	Range	0 to 50,00,000 Cr (INR)
Fundamentals	Price-to-Earnings Ratio (P/E)	Range	-100 to 500
Fundamentals	Price-to-Book Ratio (P/B)	Range	0 to 100
Fundamentals	Dividend Yield (%)	Range	0 to 25
Fundamentals	Earnings Per Share (EPS)	Range	-500 to 5000
Fundamentals	Return on Equity (ROE %)	Range	-100 to 200
Fundamentals	Return on Capital Employed (ROCE %)	Range	-100 to 200
Fundamentals	Debt-to-Equity Ratio	Range	0 to 10
Fundamentals	Current Ratio	Range	0 to 20
Fundamentals	Promoter Holding (%)	Range	0 to 100
Fundamentals	Revenue Growth (YoY %)	Range	-100 to 500
Fundamentals	Profit Growth (YoY %)	Range	-100 to 1000
Market Data	Last Traded Price	Range	0 to 500,000
Market Data	52-Week High Proximity (%)	Range	0 to 100
Market Data	52-Week Low Proximity (%)	Range	0 to 100

Market Data	Average Volume (20D)	Range	0 to 100,000,000
Market Data	Beta	Range	-2 to 5
Market Data	Day Change (%)	Range	-20 to 20
Classification	Sector	Multi-Select	IT, Banking, Pharma, Auto, FMCG, Metal, Energy, Realty, Telecom, Infrastructure, Media, Others
Classification	Industry	Multi-Select	Derived from Sector selection
Classification	Market Cap Category	Multi-Select	Large Cap, Mid Cap, Small Cap, Micro Cap
Classification	Index Membership	Multi-Select	NIFTY 50, NIFTY Next 50, NIFTY Midcap 100, NIFTY Smallcap 250, BSE Sensex
Technical	RSI (14)	Range	0 to 100
Technical	MACD Signal	Select	Bullish Crossover, Bearish Crossover, Neutral
Technical	Price vs SMA 50	Select	Above, Below
Technical	Price vs SMA 200	Select	Above, Below
Technical	Bollinger Band Position	Select	Above Upper, Within Bands, Below Lower
Technical	Average True Range (ATR)	Range	0 to 500
Technical	Volume vs 20D Average	Select	Above Average, Below Average, 2x Above, 3x Above
Custom	Watchlist Only	Boolean	Yes / No
Custom	Recently Updated (WebSocket)	Boolean	Yes / No

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

A6.2 Filter Execution Pipeline

The filter engine must follow this execution pipeline:

1. **Parse:** Convert the UI filter configuration into a normalised filter AST (Abstract Syntax Tree) where each node is either an AND group, an OR group, or a leaf predicate.
2. **Optimise:** Reorder predicates by selectivity (most restrictive first). Numeric range filters are typically more selective than category filters and should execute first.
3. **Execute:** Iterate through the stock universe and apply the predicate chain. Use short-circuit evaluation: if any AND predicate fails, skip remaining predicates for that stock.
4. **Sort:** Apply the active sort configuration to the filtered results. Use a stable sort algorithm to maintain relative order when sort values are equal.
5. **Paginate:** Slice the sorted results for the current virtual scroll viewport.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A7: TESTING STRATEGY AND REQUIREMENTS

The intern must implement a comprehensive test suite covering unit tests, integration tests, and performance tests. The minimum coverage requirement is 70 percent.

A7.1 Testing Framework Stack

Layer	Tool	Purpose
Unit Tests	Vitest	Component logic, hooks, utility functions, indicator calculations
Component Tests	React Testing Library	Component rendering, user interaction, accessibility
Integration Tests	Playwright or Cypress	End-to-end filter workflows, chart interactions, WebSocket scenarios
Performance Tests	Custom benchmarks	Filter execution time, scroll FPS, memory profiling
Visual Regression	Storybook + Chromatic (optional)	Component visual consistency across changes

A7.2 Test Examples

```
// __tests__/indicators/sma.test.ts
import { describe, it, expect } from 'vitest';
import { calculateSMA } from '@/lib/indicators';

describe('Simple Moving Average', () => {
  it('calculates SMA correctly for a basic dataset', () => {
    const prices = [10, 12, 14, 16, 18, 20, 22, 24, 26, 28];
    const sma5 = calculateSMA(prices, 5);
    expect(sma5[4]).toBeCloseTo(14); // (10+12+14+16+18)/5
    expect(sma5[9]).toBeCloseTo(24); // (20+22+24+26+28)/5
  });

  it('returns undefined for periods with insufficient data', ()
=> {
    const prices = [10, 12, 14];
    const sma5 = calculateSMA(prices, 5);
    expect(sma5[0]).toBeUndefined();
    expect(sma5[2]).toBeUndefined();
  });

  it('handles single-element period', () => {
    const prices = [42, 43, 44];
    const sma1 = calculateSMA(prices, 1);
    expect(sma1[0]).toBe(42);
```

```
    expect(sma1[2]).toBe(44);
  });
});

// __tests__/filters/filterEngine.test.ts
import { describe, it, expect } from 'vitest';
import { filterStocks } from '@/lib/filterEngine';
import { generateMockStocks } from '@/test-utils/mockData';

describe('Filter Engine Performance', () => {
  const stocks = generateMockStocks(5000);

  it('filters 5000 stocks in under 200ms', () => {
    const filters = [
      { field: 'marketCap', operator: 'gte', value: 10000 },
      { field: 'pe', operator: 'between', value: [10, 30] },
          { field: 'sector', operator: 'in', value: ['IT',
'Banking'] },
    ];
    const start = performance.now();
    const result = filterStocks(stocks, filters);
    const elapsed = performance.now() - start;
    expect(elapsed).toBeLessThan(200);
    expect(result.length).toBeGreaterThan(0);
  });
});
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A8: MANDATORY LEARNING RESOURCES

The intern must study the following resources before and during the project.

A8.1 Official Documentation

- React 18 Documentation: https://react.dev/learn
- Next.js 14 App Router: https://nextjs.org/docs/app
- TanStack Table v8: https://tanstack.com/table/latest
- TanStack Virtual: https://tanstack.com/virtual/latest
- Lightweight Charts: https://tradingview.github.io/lightweight-charts/
- Zustand: https://zustand.docs.pmnd.rs/getting-started/introduction
- React Query (TanStack Query): https://tanstack.com/query/latest
- Tailwind CSS: https://tailwindcss.com/docs/installation/using-vite
- Vitest: https://vitest.dev/guide/

A8.2 Reference Applications (Study, Do Not Copy)

The intern must study the following applications to understand the expected level of quality. Copying code from these applications is strictly prohibited and will result in immediate disqualification.

- Screener.in: https://www.screener.in/ (Indian equity screener with fundamental filters)
- Finviz: https://finviz.com/screener.ashx (US equity screener with technical filters and charts)
- TradingView Screener: https://www.tradingview.com/screener/ (Global equity screener with advanced charting)
- Trendlyne: https://trendlyne.com/stock-screeners/ (Indian market screener with DVM scores)
- Chartink: https://chartink.com/screener (Indian technical screener with scan builder)

SECTION A9: MOCK API SPECIFICATION

The application must be backed by a mock API layer that simulates production-grade API behaviour. The intern must implement the following endpoints:

A9.1 REST API Endpoints

Endpoint	Method	Description	Response
GET /api/stocks	GET	Fetch the entire stock universe with all fields	Array of Stock objects (5,000+)
GET /api/stocks/:symbol	GET	Fetch detailed data for a single stock	Stock object with extended fields
GET /api/stocks/:symbol/history	GET	Fetch OHLCV history for a stock	Array of OHLCV candles (252 days minimum)
GET /api/stocks/:symbol/fundamentals	GET	Fetch detailed fundamental data	Balance sheet, P&L, ratios
GET /api/filters/presets	GET	Fetch saved filter presets	Array of FilterPreset objects
POST /api/filters/presets	POST	Save a custom filter preset	Created FilterPreset object
GET /api/sectors	GET	Fetch sector and industry classification tree	Nested sector-industry structure
GET /api/indices	GET	Fetch index composition data	Array of Index objects with constituents

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

A9.2 API Response Format

All API responses must follow a consistent envelope format:

```
// Standard API response envelope
interface ApiResponse<T> {
  success: boolean;
  data: T;
  meta: {
    total: number;          // Total records available
    page: number;           // Current page (for paginated
endpoints)
    pageSize: number;       // Records per page
    timestamp: string;      // ISO 8601 timestamp
    executionTimeMs: number; // Server-side execution time
  };
  error?: {
    code: string;           // Machine-readable error code
    message: string;        // Human-readable error description
  };
}

// Example usage in API route
// src/app/api/stocks/route.ts
import { NextResponse } from 'next/server';
import { generateMockStocks } from '@/lib/mockDataGenerator';

const stockCache = new Map<string, { data: Stock[]; timestamp:
number }>();
const CACHE_TTL = 5 * 60 * 1000; // 5 minutes

export async function GET(request: Request) {
  const start = performance.now();
  const { searchParams } = new URL(request.url);
  const page = parseInt(searchParams.get('page') || '1');
  const pageSize = parseInt(searchParams.get('pageSize') ||
'5000');

  // Check cache
  const cached = stockCache.get('universe');
  let stocks: Stock[];
  if (cached && Date.now() - cached.timestamp < CACHE_TTL) {
    stocks = cached.data;
  } else {
    stocks = generateMockStocks(5000);
    stockCache.set('universe', { data: stocks, timestamp:
Date.now() });
  }

  const paginated = stocks.slice((page - 1) * pageSize, page *
pageSize);
  return NextResponse.json({
    success: true,
    data: paginated,
```

```
    meta: {
      total: stocks.length,
      page, pageSize,
      timestamp: new Date().toISOString(),
      executionTimeMs: Math.round(performance.now() - start),
    },
  });
}
```

A9.3 OHLCV Data Generation

The historical OHLCV (Open, High, Low, Close, Volume) data generator must produce realistic candlestick data:

```
// lib/ohlcvGenerator.ts
export interface OHLCV {
  time: number;   // Unix timestamp
  open: number;
  high: number;
  low: number;
  close: number;
  volume: number;
}

export function generateOHLCV(
  startPrice: number,
  days: number = 252,  // 1 trading year
  volatility: number = 0.02,
  avgVolume: number = 1000000
): OHLCV[] {
  const candles: OHLCV[] = [];
  let currentPrice = startPrice;
  const startDate = new Date();
  startDate.setDate(startDate.getDate() - days);

  for (let i = 0; i < days; i++) {
    const date = new Date(startDate);
    date.setDate(date.getDate() + i);

    // Skip weekends
    if (date.getDay() === 0 || date.getDay() === 6) continue;

    const dailyReturn = normalRandom() * volatility;
    const open = currentPrice;
    const intraday1 = open * (1 + normalRandom() * volatility *
0.5);
    const intraday2 = open * (1 + normalRandom() * volatility *
0.5);
    const close = open * (1 + dailyReturn);
    const high = Math.max(open, close, intraday1, intraday2) *
(1 + Math.abs(normalRandom()) * 0.005);
    const low = Math.min(open, close, intraday1, intraday2) *
(1 - Math.abs(normalRandom()) * 0.005);
```

```
    // Volume is correlated with price movement magnitude
    const volumeMultiplier = 1 + Math.abs(dailyReturn) * 10;
    const volume = Math.round(avgVolume * volumeMultiplier *
(0.5 + Math.random()));

    candles.push({
      time: Math.floor(date.getTime() / 1000),
      open: round2(open), high: round2(high),
      low: round2(low), close: round2(close),
      volume,
    });
    currentPrice = close;
  }
  return candles;
}

function round2(n: number): number { return Math.round(n * 100)
/ 100; }
function normalRandom(): number {
  return Math.sqrt(-2 * Math.log(Math.random())) * Math.cos(2 *
Math.PI * Math.random());
}
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A10: ACCESSIBILITY AND INCLUSIVE DESIGN

Building an accessible financial application is not optional. Financial data must be available to all users including those who rely on assistive technologies. This section details the accessibility requirements that will be evaluated.

A10.1 Data Grid Accessibility

The data grid must implement the WAI-ARIA grid pattern:

```
// components/DataGrid/AccessibleGrid.tsx
// The grid container must have these ARIA attributes:
<div
  role="grid"
  aria-label="Stock Screener Results"
  aria-rowcount={totalRows}
  aria-colcount={columns.length}
  aria-describedby="grid-status"
>
  {/* Status announcement for screen readers */}
  <div id="grid-status" role="status" aria-live="polite"
className="sr-only">
    Showing {filteredCount} of {totalCount} stocks.
    Sorted by {sortColumn} {sortDirection}.
  </div>

  {/* Column headers */}
  <div role="row" aria-rowindex={1}>
    {columns.map((col, i) => (
      <div
        key={col.id}
        role="columnheader"
        aria-colindex={i + 1}
        aria-sort={col.isSorted ? (col.isSortedDesc ?
'descending' : 'ascending') : 'none'}
        tabIndex={0}
        onKeyDown={(e) => {
          if (e.key === 'Enter' || e.key === ' ') {
            e.preventDefault();
            col.toggleSorting();
          }
        }}
      >
        {col.header}
      </div>
    ))}
  </div>

  {/* Data rows */}
  {virtualRows.map((row) => (
    <div role="row" aria-rowindex={row.index + 2}
key={row.index}>
      {row.cells.map((cell, i) => (
        <div
```

```
            role="gridcell"
            aria-colindex={i + 1}
            tabIndex={-1}
          >
            {cell.render()}
          </div>
        ))}
      </div>
    ))}
</div>
```

A10.2 Chart Accessibility

Canvas-based charts are inherently inaccessible. The intern must provide alternative access to chart data:

- **Data Table Toggle:** A button that switches between the visual chart and an accessible data table showing the same OHLCV data in tabular format.
- **Screen Reader Announcements:** When a new data point arrives via WebSocket, announce the update: "RELIANCE: Price updated to 2,450.75, up 1.2 percent."
- **Keyboard Chart Navigation:** Arrow keys to move the crosshair left and right through candles. Press Enter to announce the OHLCV values for the focused candle.
- **High Contrast Mode:** Provide a high contrast colour scheme option where up candles are cyan and down candles are magenta, avoiding red-green which is problematic for colour-blind users.

A10.3 Filter Panel Accessibility

- Every filter control must have a visible label connected via htmlFor/id or aria-labelledby.
- Range sliders must announce their current value when adjusted (aria-valuenow, aria-valuemin, aria-valuemax).
- Multi-select dropdowns must announce the number of selected items.
- Active filter chips must be in a live region so screen readers announce when filters are added or removed.
- The "Clear All Filters" button must trigger an announcement: "All filters cleared. Showing 5,000 stocks."

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A11: COMMON PITFALLS AND ANTI-PATTERNS

The following section documents common mistakes that previous interns have made. Study these carefully to avoid repeating them.

A11.1 Performance Anti-Patterns

Anti-Pattern	Symptom	Correct Approach
Rendering all 5,000 rows in the DOM	Browser freezes on initial load, > 5 second LCP	Use TanStack Virtual to render only visible rows + overscan buffer
Creating new objects in render	Infinite re-renders, high CPU usage	Use useMemo and useCallback to stabilise object references
Filtering inside the render function	UI stutters when typing in filter inputs	Debounce filter input (300ms) and compute filtered results in useMemo
Updating state on every WebSocket message	React batching overwhelmed, dropped frames	Buffer updates in a Map and flush with requestAnimationFrame
Not memoising cell components	Entire grid re-renders when one cell updates	Wrap cell components in React.memo with shallow comparison
Loading all chart data upfront	Large initial bundle, slow page load	Lazy-load chart library with dynamic import(), load data on demand
Using setInterval for price simulation	Timer drift, inconsistent update frequency	Use setTimeout with self-correcting drift compensation or Web Workers
String concatenation for className	Unnecessary re-renders from new string references	Use clsx or tailwind-merge for stable className strings

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

A11.2 Architecture Anti-Patterns

Anti-Pattern	Problem	Correct Approach
Single monolithic store	Every state change triggers all subscriptions	Split into domain stores (filterStore, priceStore, uiStore) or use selectors
Prop drilling through 5+ levels	Tight coupling, refactoring is painful	Use Zustand selectors or React Context for cross-cutting concerns
Mixing server and client state	Stale data, cache invalidation bugs	Use React Query for server state, Zustand for client state
No error boundaries	One component crash kills the entire app	Wrap each feature area in its own ErrorBoundary with fallback UI
Using useEffect for data transformation	Extra renders, stale closure bugs	Use useMemo for derived data, useCallback for event handlers
Direct DOM manipulation in React	State and DOM out of sync, memory leaks	Always update through React state; refs only for imperative chart APIs

A11.3 Deliberate Errors in This Document

IMPORTANT: This document contains three deliberate technical errors in the code examples. The intern must identify these errors, explain why they are wrong, and document the corrections in a file called ERRATA.md in the repository. Finding all three errors earns 15 bonus points (5 per error). Hint: one is in the indicator calculation, one is in the WebSocket reconnection logic, and one is in the TypeScript type definitions.

SECTION A12: TAILWIND CSS DESIGN SYSTEM SPECIFICATION

The application must use a consistent design system built on Tailwind CSS. The following design tokens are mandatory:

A12.1 Colour Palette

```
// tailwind.config.ts
import type { Config } from 'tailwindcss';

const config: Config = {
  content: ['./src/**/*.{js,ts,jsx,tsx,mdx}'],
  darkMode: 'class',
  theme: {
    extend: {
      colors: {
        // Brand
        brand: {
          50: '#E8F4FD', 100: '#C5E3FA', 200: '#9DD0F6',
          500: '#2563EB', 600: '#1D4ED8', 700: '#1E40AF',
          900: '#1E3A5F',
        },
        // Semantic
        positive: { light: '#DCFCE7', DEFAULT: '#22C55E', dark:
'#166534' },
        negative: { light: '#FEE2E2', DEFAULT: '#EF4444', dark:
'#991B1B' },
        warning: { light: '#FEF3C7', DEFAULT: '#F59E0B', dark:
'#92400E' },
        // Data visualisation
        chart: {
          candle: { up: '#22C55E', down: '#EF4444' },
          sma20: '#3B82F6', sma50: '#F97316', sma200:
'#8B5CF6',
          ema12: '#06B6D4', ema26: '#EC4899',
          bollinger: { fill: '#3B82F620', line: '#3B82F6' },
          rsi: '#8B5CF6',
          volume: '#94A3B8',
        },
      },
      fontFamily: {
        sans: ['Inter', 'system-ui', 'sans-serif'],
        mono: ['JetBrains Mono', 'Fira Code', 'monospace'],
      },
      animation: {
        'flash-green': 'flashGreen 300ms ease-out',
        'flash-red': 'flashRed 300ms ease-out',
        'slide-in': 'slideIn 200ms ease-out',
      },
      keyframes: {
        flashGreen: { '0%': { backgroundColor: '#DCFCE7' },
'100%': { backgroundColor: 'transparent' } },
        flashRed: { '0%': { backgroundColor: '#FEE2E2' },
'100%': { backgroundColor: 'transparent' } },
```

```
        slideIn: { '0%': { transform: 'translateX(-10px)',
opacity: '0' }, '100%': { transform: 'translateX(0)', opacity:
'1' } },
      },
    },
  },
  plugins: [],
};

export default config;
```

A12.2 Component Styling Conventions

The intern must follow these styling conventions:

- Grid container: bg-white dark:bg-gray-900 rounded-xl shadow-sm border border-gray-200 dark:border-gray-800
- Grid header cells: text-xs font-semibold uppercase tracking-wider text-gray-500 dark:text-gray-400
- Grid data cells: text-sm font-mono tabular-nums for all numeric values
- Positive values: text-positive dark:text-positive-light
- Negative values: text-negative dark:text-negative-light
- Filter panel: w-80 border-r border-gray-200 dark:border-gray-800 overflow-y-auto
- Buttons: All interactive elements must have focus-visible:ring-2 focus-visible:ring-brand-500 focus-visible:ring-offset-2
- Transitions: All hover and state transitions must use transition-colors duration-150

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

A12.3 Responsive Breakpoints

Breakpoint	Width	Layout	Key Changes
Mobile	< 768px	Single column, stacked	Filter panel as full-screen overlay, no chart side-panel, simplified grid with fewer columns
Tablet	768px - 1279px	Stacked with collapsible sidebar	Filter panel toggles via hamburger, chart below grid, medium column set
Desktop	1280px - 1535px	Side-by-side	Filter sidebar + grid + chart panel, all columns visible
Wide	>= 1536px	Full layout	Wider chart panel, optional secondary data pane, all features visible

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A13: REPOSITORY STRUCTURE AND NAMING CONVENTIONS

The following naming conventions are mandatory for the submitted repository:

A13.1 File Naming

- React components: PascalCase (StockChart.tsx, FilterPanel.tsx, PriceCell.tsx)
- Hooks: camelCase with use prefix (useStockScreener.ts, useWebSocket.ts, useKeyboardNav.ts)
- Utility functions: camelCase (filterEngine.ts, mockDataGenerator.ts, indicatorCalcs.ts)
- Type definitions: camelCase (stock.ts, filters.ts, chart.ts)
- Test files: Same name as source with .test suffix (filterEngine.test.ts, PriceCell.test.tsx)
- Constants: UPPER_SNAKE_CASE in a constants/ directory (SECTORS.ts, FILTER_PRESETS.ts)

A13.2 Git Commit Message Format

All commits must follow the Conventional Commits specification:

```
// Format: <type>(<scope>): <description>
// Examples:
feat(grid): implement virtual scrolling with TanStack Virtual
feat(filters): add range filter component with dual-handle
slider
feat(chart): integrate Lightweight Charts with candlestick
series
feat(ws): implement WebSocket simulation with geometric
Brownian motion
fix(grid): resolve blank rows during fast scroll
perf(filters): optimise predicate chain with short-circuit
evaluation
test(indicators): add unit tests for SMA and EMA calculations
docs(readme): add architecture diagram and setup instructions
refactor(store): split monolithic store into domain slices
style(grid): apply consistent cell formatting for currency
values
chore(deps): update TanStack Table to v8.12.0
```

A13.3 Branch Strategy

The intern must use a feature branch workflow:

- main: Production-ready code. Only merge via pull request (self-merge is acceptable for solo projects but PR must exist for audit trail).
- feat/*: Feature branches (feat/virtual-scrolling, feat/filter-engine, feat/candlestick-chart).
- fix/*: Bug fix branches.
- The repository must have a minimum of 7 merged pull requests (one per day) at the time of transfer.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

Note on the Simulation Concept: The gamified simulation concept presented in this section is provided as a reference framework. Students may choose to follow this concept or design their own original gamified simulation - with a different name, structure, narrative, or game mechanics - as long as it meets the assessment criteria outlined in Part F.

Simulation Title: "SCREENER WARS: The Brokerage Battle"

SECTION B1: SCENARIO NARRATIVE

You are the newly appointed Lead Front-End Developer at EquityPulse, a Series-A funded retail brokerage startup headquartered in Mumbai with ambitions to disrupt the Indian equity screening market. The firm has secured INR 25 crore in funding and your mandate is clear: build a stock screening platform that is faster, more intuitive, and more feature-rich than Screener.in and Trendlyne within seven days for an internal demo to the board of directors and lead investors.

The CTO (your direct report) has given you a mock API specification, a dataset of 5,000 NSE and BSE listed stocks with fundamental and technical data, and a WebSocket endpoint specification for simulated real-time price feeds. The board demo is non-negotiable. Missing the deadline means the next funding round is at risk. Shipping a buggy, slow, or poorly designed product means the CTO loses credibility. You must deliver production-quality software under extreme time pressure.

Your competitors are not sleeping. Two other interns in the cohort are building their own versions of the screener (the Arena Mode). The board will compare all submissions side by side and rank them. Only the top submission gets featured on the company's investor deck.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION B2: SIMULATION MODES

B2.1 Campaign Mode (Solo Development)

The primary mode. The intern works alone to build the complete application over seven days. Progress is tracked daily with mandatory commits and deployments.

Level	Title	Day	Objective	Points
1	Foundation Sprint	Day 1-2	Project setup, component architecture, mock data layer, basic grid rendering	100
2	Filter Forge	Day 3	Implement 30+ filters with combinatorial logic, filter panel UI, predicate engine	200
3	Chart Champion	Day 4	Candlestick chart with 5 indicators, timeframe switching, responsive layout	200
4	WebSocket Warrior	Day 5	Real-time price simulation, batch updates, flash animations, reconnection logic	150
5	Performance Purist	Day 6	Virtual scrolling, memoisation, bundle optimisation, Lighthouse audit	200
6	Final Boss: The Board Demo	Day 7	Documentation, testing, deployment, GitHub transfer, final polish	150

B2.2 Arena Mode (Competitive Evaluation)

All interns in the Frontend Developer cohort submit their projects for comparative evaluation. The UX Review Panel (comprising Zetheta mentors) conducts a blind evaluation across the following criteria:

- Visual Design Quality: Layout consistency, typography, colour palette, responsive behaviour.
- Feature Completeness: Number of working filters, chart indicators, keyboard shortcuts.
- Performance: Lighthouse score comparison, filter response time benchmarks, scroll FPS.
- Code Quality: Static analysis results, test coverage, architectural patterns.

- Innovation: Unique features not specified in the requirements (dark mode, saved screeners, AI-powered suggestions).

B2.3 Sandbox Mode (Extension Challenges)

For interns who complete the core requirements ahead of schedule, the following optional challenges are available for bonus points:

- **Challenge 1 - Heatmap View (+30 points):** Implement a market heatmap similar to Finviz's map view, where stocks are sized by market cap and coloured by day change percentage.
- **Challenge 2 - Saved Screeners (+25 points):** Allow users to save filter configurations with custom names and load them later. Persist using localStorage or IndexedDB.
- **Challenge 3 - Export to CSV/PDF (+20 points):** Export the filtered results to CSV and the chart to PDF with a professional layout.
- **Challenge 4 - Dark Mode (+15 points):** Full dark mode implementation using CSS custom properties with smooth transition animation.
- **Challenge 5 - Keyboard Power User (+20 points):** Full keyboard navigation: Arrow keys for grid navigation, / to focus search, F for filter panel toggle, C for chart toggle, Escape to dismiss modals.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION B3: ACHIEVEMENT BADGES

Interns earn achievement badges that are recorded on their Zetheta performance profile:

Badge	Requirement	Rarity
Speed Demon	Filter response time < 100ms for 5,000 stocks	Legendary
Pixel Perfect	Zero layout shift (CLS = 0) on all viewport sizes	Epic
Test Titan	90%+ code coverage with meaningful tests (no snapshot-only)	Epic
Chart Wizard	All 5 indicators implemented with correct mathematical formulas	Rare
WebSocket Sage	Zero dropped messages, < 50ms render latency, auto-reconnect working	Rare
Bundle Boss	Total JS bundle < 250KB gzipped	Epic
Accessibility Ace	Lighthouse Accessibility score > 95	Rare
Git Guru	50+ meaningful commits with conventional commit messages	Common
Documentation Champion	README with architecture diagrams, API docs, setup guide	Common
Innovation Star	Implemented 3+ sandbox challenges	Legendary

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION B4: SCORING SYSTEM AND LEADERBOARD

The gamified simulation uses a 1,000-point scoring system that directly maps to the rubric in Part F. Points are awarded progressively as the intern completes each Campaign Mode level:

B4.1 Point Accumulation Schedule

Day	Level Completed	Points Available	Cumulative Maximum	Milestone Reward
Day 1-2	Level 1: Foundation Sprint	100	100	"Apprentice" badge unlocked; access to Architecture Review channel
Day 3	Level 2: Filter Forge	200	300	"Journeyman" badge unlocked; access to Code Review feedback from senior mentor
Day 4	Level 3: Chart Champion	200	500	"Specialist" badge unlocked; midpoint performance snapshot shared
Day 5	Level 4: WebSocket Warrior	150	650	"Expert" badge unlocked; eligible for Arena Mode bonus points
Day 6	Level 5: Performance Purist	200	850	"Master" badge unlocked; eligible for Sandbox Mode challenges
Day 7	Level 6: Final Boss	150	1,000	"Champion" badge unlocked; eligible for distinction grade

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

B4.2 Arena Mode Scoring (Competitive Bonus)

In Arena Mode, all submissions within the same cohort are ranked blind. Bonus points are awarded based on relative ranking:

- **Rank 1 (Gold):** +50 bonus points. Featured on the Zetheta investor deck. Personal recommendation letter from the CTO.
- **Rank 2 (Silver):** +30 bonus points. Featured on the Zetheta engineering blog. Priority for advanced project selection.
- **Rank 3 (Bronze):** +20 bonus points. Certificate of Excellence. Priority for team lead track.
- **All Other Submissions:** No bonus points. Standard assessment based on rubric only.

Arena Mode evaluation is conducted by a three-member UX Review Panel using a standardised evaluation matrix. Each panelist scores independently. The final Arena rank is determined by the mean score across all three evaluations.

B4.3 Sandbox Mode Scoring

Sandbox challenges provide bonus points that can push a submission beyond the 1,000-point ceiling. However, sandbox points are only awarded if the core Campaign Mode requirements achieve at least 500 points. An intern who implements three sandbox challenges but has a broken filter engine will not receive sandbox bonus points.

Challenge	Bonus Points	Difficulty	Estimated Time	Prerequisites
Heatmap View	+30	Advanced	4-6 hours	D3.js or custom Canvas rendering; complete grid and filter engine
Saved Screeners	+25	Intermediate	2-3 hours	Working filter engine; IndexedDB or localStorage
Export to CSV/PDF	+20	Intermediate	2-3 hours	Working filter engine and grid; file generation library
Dark Mode	+15	Beginner	1-2 hours	CSS custom properties; Tailwind dark mode class strategy
Keyboard Power User	+20	Advanced	3-4 hours	Complete grid and chart; focus management expertise

SECTION B5: SIMULATION DATA ARCHITECTURE

The mock data universe simulates the Indian equity market with the following distribution:

Category	Count	Percentage	Market Cap Range (INR Cr)	Example Companies
Large Cap	100	2%	50,000 - 20,00,000	Reliance, TCS, HDFC Bank, Infosys, ICICI Bank
Mid Cap	400	8%	10,000 - 49,999	Indian Hotels, Ashok Leyland, Voltas, Persistent Systems
Small Cap	1,500	30%	1,000 - 9,999	Gravita India, Route Mobile, Cera Sanitaryware
Micro Cap	3,000	60%	50 - 999	Simulated companies with realistic fundamental profiles

B5.1 Sector Distribution

Sector	Stock Count	Weight	Avg P/E	Avg Beta	Avg Dividend Yield
Banking & Financial Services	750	15%	18.5	1.10	1.2%
Information Technology	600	12%	25.0	0.85	1.5%
Pharmaceuticals & Healthcare	500	10%	30.0	0.70	0.8%
Consumer Goods (FMCG)	400	8%	35.0	0.60	1.8%
Automobile & Auto Components	350	7%	22.0	1.05	1.0%
Metals & Mining	300	6%	12.0	1.40	2.5%
Energy (Oil, Gas, Power)	300	6%	14.0	0.95	3.0%

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Real Estate & Construction	350	7%	20.0	1.30	0.5%
Telecom & Media	200	4%	28.0	0.90	0.3%
Infrastructure & Capital Goods	400	8%	18.0	1.15	1.0%
Chemicals & Fertilisers	350	7%	22.0	0.80	1.2%
Others (Textiles, Paper, etc.)	500	10%	16.0	1.00	1.5%

B5.2 Realistic Data Correlation Rules

The mock data generator must enforce the following correlations to produce realistic data. Submissions with obviously random data that violates these rules will be penalised:

- **Market Cap vs Beta:** Large-cap stocks (> 50,000 Cr) must have beta between 0.5 and 1.2. Micro-cap stocks can have beta between 0.3 and 2.5. The correlation coefficient should be approximately -0.3.
- **Market Cap vs Promoter Holding:** Large-cap promoter holding typically ranges from 40% to 75%. Micro-cap promoter holding ranges from 20% to 90% with higher variance.
- **P/E vs Growth:** High-growth companies (revenue growth > 25%) should have higher P/E ratios (20-80). Slow-growth or declining companies should have lower P/E (5-15) or negative P/E.
- **Debt/Equity vs Sector:** Banking and NBFC stocks should have high debt/equity (5-15). IT and FMCG stocks should have low debt/equity (0-0.5). Infrastructure stocks should have moderate debt/equity (0.5-2.0).
- **RSI vs Recent Price Movement:** Stocks with positive 5-day returns should have RSI trending above 50. Stocks with negative 5-day returns should have RSI trending below 50. RSI should never be exactly 50 for all stocks.
- **Volume vs Price Movement:** Stocks with large absolute daily change should have above-average volume. This correlation must be evident in the generated data.

SECTION B6: WEEKLY PROGRESS REPORT TEMPLATE

At the end of each day, the intern must commit a progress report to the repository in a /progress directory following this template:

```
// progress/day-01.md
# Day 1 Progress Report
## Date: [YYYY-MM-DD]
## Intern: [Full Name]

## Tasks Completed
- [x] Task 1.1: Project Initialisation
- [x] Task 1.2: TypeScript Type Definitions
- [x] Task 1.3: Mock Data Generator
- [x] Task 1.4: Basic Data Grid Implementation
- [ ] Task 1.5: Zustand Store Setup (partially complete)
- [x] Task 1.6: Architecture Documentation

## Hours Worked: 8
## Commits Made: 7

## Key Decisions
- Chose Zustand over Redux because: [reasoning]
- Used fixed row height (36px) for virtual scrolling because:
[reasoning]

## Blockers / Challenges
- [Describe any technical challenges encountered]
- [Describe how you resolved them or plan to resolve them]

## AI Tools Used Today
- Claude: Generated initial type definitions (modified 40% of
output)
- GitHub Copilot: Autocomplete for repetitive column
definitions

## Tomorrow's Plan
- Complete Zustand store setup
- Implement virtual scrolling
- Begin keyboard navigation

## Self-Assessment (1-5): 4
## Confidence Level for On-Time Completion (1-5): 4
```

PART C: CASE STUDIES AND REAL-WORLD APPLICATIONS

The following case studies illustrate real-world challenges in building financial screening applications. The intern must reference at least two of these case studies in their technical report and explain how their architecture addresses the identified challenges.

C1. Case Study: Zerodha Kite - Scaling Real-Time Data for 10 Million Users

Zerodha, India's largest discount brokerage, serves over 10 million active users through its Kite trading platform. Their engineering team faced significant challenges scaling real-time market data distribution.

Key Technical Challenges:

- Distributing real-time tick data for 4,000+ instruments to millions of concurrent WebSocket connections.
- Maintaining sub-100ms latency from exchange feed to user screen across varying network conditions.
- Managing memory efficiently on the client when displaying large watchlists and order books.

Architecture Decisions:

- Binary WebSocket protocol (custom protocol buffer format) instead of JSON to reduce bandwidth by 70%.
- Client-side data normalisation layer that batches incoming ticks and updates the DOM at 60 FPS using requestAnimationFrame.
- Lazy loading of chart components: charts are only initialised when the user explicitly opens a chart for a specific instrument.

Relevance to project:

The intern should study Zerodha's approach to WebSocket message batching and requestAnimationFrame-based rendering. While the project uses JSON instead of protocol buffers, the batching and throttling principles are directly applicable.

C2. Case Study: Screener.in - Building India's Most Popular Equity Screener

Screener.in is a free-to-use equity screening platform that serves over 2 million monthly active users. Built by a small team, it demonstrates how a well-designed filter engine can handle complex queries efficiently.

Key Technical Challenges:

- Supporting 100+ filter parameters with arbitrary combinatorial logic (AND, OR, nested groups).
- Rendering financial data tables with inline sparkline charts for 5,000+ stocks.
- Handling complex calculated fields (e.g., 5-year CAGR, Piotroski F-Score) that require multiple data points.

Architecture Insights:

- Server-side filtering with precomputed indices for common filter combinations.
- Progressive data loading: basic data loads first, followed by derived metrics on demand.
- Custom query language that allows power users to write expressions like: "Market Cap > 1000 AND ROE > 15 AND Debt/Equity < 0.5".

Relevance to project:

The intern's filter engine should aspire to Screener.in's expressiveness while achieving client-side execution. The custom query language concept is a sandbox challenge worth implementing for bonus points.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

C3. Case Study: TradingView - Real-Time Charting at Scale

TradingView serves over 50 million monthly active users and is the gold standard for web-based financial charting. Their Lightweight Charts library (which the intern is recommended to use) is the open-source version of their internal charting engine.

Key Technical Challenges:

- Rendering millions of data points on a single chart without performance degradation.
- Supporting 100+ technical indicators with real-time recalculation as new data arrives.
- Cross-browser compatibility and consistent rendering across devices and screen sizes.

Architecture Insights:

- Canvas-based rendering (not SVG) for performance with large datasets.
- Level-of-detail rendering: fewer data points are rendered when the chart is zoomed out.
- Indicator calculation runs in a Web Worker to avoid blocking the main thread.

Relevance to project:

The intern should study how Lightweight Charts handles real-time updates and consider running indicator calculations in a Web Worker for bonus points. The level-of-detail concept is relevant for the volume profile indicator.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

C4. Case Study: Finviz - UX Design Patterns for Complex Data

Finviz is a US equity screening platform known for its information density and efficient UX design. Despite displaying massive amounts of data, the interface remains navigable.

Key UX Design Patterns:

- Compact data grid with fixed headers, horizontal scrolling, and column pinning for the symbol column.
- Heatmap view that provides an instant visual overview of market performance by sector and market cap.
- Filter panel that shows the count of matching stocks in real time as the user adjusts criteria.
- Quick sort by clicking column headers with visual sort direction indicators.

Relevance to project:

The intern must implement column pinning, fixed headers, and real-time filter count. The heatmap view is a sandbox challenge. Finviz's information density should be the design aspiration.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

C5. Case Study: Bloomberg Terminal - Keyboard-First Design Philosophy

The Bloomberg Terminal, used by over 325,000 financial professionals globally, is the benchmark for information-dense financial applications. Its keyboard-first design philosophy is a critical lesson for any developer building financial tools.

Key Design Principles:

- **Keyboard Supremacy:** Professional traders never lift their hands from the keyboard. Every Bloomberg function is accessible via a keyboard command. The terminal's entire navigation is built around typed commands (e.g., AAPL <Equity> <GO> to view Apple equity data).
- **Information Density:** Bloomberg displays up to 50 data points per row in its equity screening function. Colour coding, compact fonts, and minimal whitespace maximise the amount of information visible without scrolling.
- **Contextual Defaults:** Every screen loads with intelligent defaults. The equity screener pre-populates with the user's most recently used filters. This concept of persistent user preferences dramatically reduces time to insight.
- **Progressive Disclosure:** Summary data is shown first. Detailed data loads on demand when the user drills down. This prevents information overload while ensuring comprehensive data is always one keystroke away.

Architecture Lessons:

- Bloomberg uses a thick-client architecture with aggressive local caching. Market data is streamed via proprietary protocols and cached locally for instant retrieval. The UI never waits for a network round-trip for frequently accessed data.
- The terminal implements a command palette pattern (similar to VS Code's Ctrl+Shift+P) that allows power users to navigate directly to any screen or function without using menus.
- All data tables support column customisation: users can add, remove, reorder, resize, and freeze columns. These preferences are persisted per user and per screen.

Relevance to project:

The intern should implement a command palette (Ctrl+K or /) that allows users to search for stocks, apply preset filters, and toggle features without touching the mouse. This is a strong differentiator in Arena Mode and demonstrates understanding of professional trading tool UX conventions. The persistent filter preferences concept should be implemented via localStorage or IndexedDB.

PART D: PROJECT METHODOLOGY AND DELIVERABLES

DAY 1-3: ENVIRONMENT SETUP, ARCHITECTURE DESIGN, AND MOCK DATA LAYER

Task 1.1: Project Initialisation

- Create a new Next.js 14 project with TypeScript, App Router, and Tailwind CSS.
- Configure ESLint with the Airbnb TypeScript preset and Prettier for code formatting.
- Set up Husky pre-commit hooks with lint-staged to enforce code quality on every commit.
- Install and configure: @tanstack/react-table, @tanstack/react-virtual, lightweight-charts, zustand, @tanstack/react-query, vitest, @testing-library/react.
- Create the following directory structure:

```
src/
  app/
    layout.tsx
    page.tsx
    screener/
      page.tsx
      [symbol]/
        page.tsx
  components/
    ui/           # Shared UI primitives (Button, Input, Select,
etc.)
    DataGrid/     # TanStack Table components
    Chart/        # Lightweight Charts wrapper
    FilterPanel/  # Filter controls and logic
    Layout/       # Header, Sidebar, Footer
  hooks/          # Custom React hooks
  stores/         # Zustand stores
  lib/            # Utility functions, filter engine, indicator
calculations
  types/          # TypeScript type definitions
  __tests__/      # Test files mirroring src structure
  test-utils/     # Test utilities and mock data generators
```

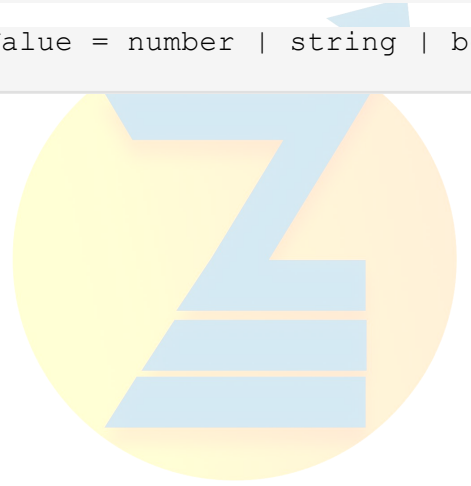
Task 1.2: TypeScript Type Definitions

Define comprehensive type definitions for the entire application:

```
// types/stock.ts
export interface Stock {
  symbol: string;
  companyName: string;
  sector: Sector;
  industry: string;
  marketCapCategory: 'Large Cap' | 'Mid Cap' | 'Small Cap' |
'Micro Cap';
  indexMembership: string[];

  // Price data
  lastPrice: number;
  previousClose: number;
  dayOpen: number;
  dayHigh: number;
  dayLow: number;
  changePercent: number;
  changeAbsolute: number;
  volume: number;
  avgVolume20D: number;
  week52High: number;
  week52Low: number;

  // Fundamentals
  marketCap: number;
  pe: number | null;
  pb: number;
  dividendYield: number;
  eps: number;
  roe: number;
  roce: number;
  debtToEquity: number;
  currentRatio: number;
  promoterHolding: number;
  revenueGrowthYoY: number;
  profitGrowthYoY: number;

  // Technical
  rsi14: number;
  sma50: number;
  sma200: number;
  beta: number;
  atr: number;
  macdSignal: 'Bullish' | 'Bearish' | 'Neutral';
  bollingerPosition: 'Above' | 'Within' | 'Below';
  volumeVsAvg: 'Below' | 'Above' | '2x' | '3x';
}

export type Sector = 'IT' | 'Banking' | 'Pharma' | 'Auto' |
'FMCG' |
```

```
  'Metal' | 'Energy' | 'Realty' | 'Telecom' | 'Infrastructure'
| 'Media' | 'Others';

export interface FilterConfig {
  id: string;
  field: keyof Stock;
  operator: FilterOperator;
  value: FilterValue;
  enabled: boolean;
}

export type FilterOperator = 'eq' | 'neq' | 'gt' | 'gte' | 'lt'
| 'lte' |
  'between' | 'in' | 'notIn' | 'contains' | 'startsWith';
```

```
export type FilterValue = number | string | boolean | number[]
| string[];
```



ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Task 1.3: Mock Data Generator

Create a mock data generator that produces 5,000 realistic stock records with correlated data (e.g., large-cap stocks should have lower beta and higher promoter holding):

```
// lib/mockDataGenerator.ts
const SECTORS: Record<Sector, { companies: string[]; avgPE:
number; avgBeta: number }> = {
  IT: { companies: ['TCS', 'Infosys', 'Wipro', 'HCL Tech',
'Tech Mahindra'],
        avgPE: 25, avgBeta: 0.8 },
  Banking: { companies: ['HDFC Bank', 'ICICI Bank', 'SBI',
'Kotak', 'Axis Bank'],
             avgPE: 18, avgBeta: 1.1 },
  Pharma: { companies: ['Sun Pharma', 'Dr Reddy', 'Cipla',
'Divi Labs', 'Aurobindo'],
            avgPE: 30, avgBeta: 0.7 },
  // ... complete all sectors
};

export function generateMockStocks(count: number = 5000):
Stock[] {
  const stocks: Stock[] = [];
  for (let i = 0; i < count; i++) {
    const sector = randomSector();
    const marketCap = generateMarketCap();
    const capCategory = categoriseByMarketCap(marketCap);
    const price = generatePrice(marketCap);
    stocks.push({
      symbol: generateSymbol(sector, i),
      companyName: generateCompanyName(sector, i),
      sector,
      industry: generateIndustry(sector),
      marketCapCategory: capCategory,
      indexMembership: assignIndices(marketCap),
      lastPrice: price,
      previousClose: price * (1 + normalRandom() * 0.02),
      // ... generate all fields with realistic correlations
    });
  }
  return stocks;
}
```

Task 1.4: Basic Data Grid Implementation

Implement the TanStack Table data grid with the following features:

- All column definitions from the type specification above.
- Column sorting by clicking headers (ascending, descending, none cycle).
- Fixed header that remains visible during vertical scrolling.
- Column resizing by dragging column borders.
- Basic styling with Tailwind CSS: alternating row colours, hover state, selected row highlight.

Task 1.5: Zustand Store Setup

Implement the complete Zustand store as specified in Section A5.1. Include middleware for devtools (development only) and persistence (watchlist only).

Task 1.6: Architecture Documentation

Create an ARCHITECTURE.md file in the repository root with:

- A component hierarchy diagram (use ASCII art or Mermaid syntax).
- A state management flow diagram showing data flow between React Query, Zustand, and component state.
- A decision log explaining why each major library was chosen.

Deliverable: Working Next.js application with a data grid displaying 5,000 mock stocks, column sorting, basic styling, Zustand store initialised, and architecture documentation committed to the repository.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

DAY 4-6: ADVANCED DATA GRID: VIRTUAL SCROLLING, COLUMN FEATURES, AND KEYBOARD NAVIGATION

Task 2.1: Virtual Scrolling Implementation

Integrate TanStack Virtual with the data grid:

- Implement row virtualisation with a fixed row height of 36 pixels.
- Configure an overscan of 10 rows above and below the viewport.
- Test scrolling performance with 5,000, 10,000, and 20,000 rows.
- Implement smooth scrollbar behaviour that accurately reflects total row count.
- Add a scroll-to-row function that can programmatically scroll to any row index.

Task 2.2: Column Pinning and Horizontal Scroll

Implement column pinning:

- The Symbol column must be pinned to the left and always visible during horizontal scrolling.
- Users must be able to pin and unpin additional columns via a right-click context menu.
- Pinned columns must have a subtle shadow separator from scrollable columns.

Task 2.3: Keyboard Navigation

Implement full keyboard navigation for the data grid:

- Arrow keys: Move the active cell highlight up, down, left, right.
- Enter: Open the stock detail view (chart) for the selected row.
- Space: Toggle the stock on/off the watchlist.
- Home/End: Jump to the first/last row.
- Page Up/Page Down: Scroll by one viewport height.
- Tab: Move focus between the filter panel, data grid, and chart area.
- All keyboard shortcuts must be discoverable via a "?" shortcut that opens a cheat sheet modal.

Task 2.4: Cell Formatting and Conditional Styling

Implement specialised cell renderers:

- PriceCell: Format with INR currency symbol, two decimal places, and thousand separators.
- ChangeCell: Green text and up arrow for positive, red text and down arrow for negative.
- VolumeCell: Abbreviate large numbers (e.g., 1.2Cr, 45L, 8K).
- MarketCapCell: Display in Crore (Cr) with appropriate abbreviation.
- RSICell: Colour-coded background (green < 30, yellow 30-70, red > 70).

Task 2.5: Unit Tests for Day 2 Features

Write tests for virtual scrolling behaviour, keyboard navigation, and cell formatters. Minimum 10 test cases.

Deliverable: Data grid with virtual scrolling, column pinning, keyboard navigation, formatted cells, and 10+ passing unit tests.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

DAY 7-9: FILTER ENGINE: 30+ CRITERIA WITH COMBINATORIAL LOGIC

Task 3.1: Filter Engine Core

Implement the filter execution pipeline as specified in Section A6.2:

- Build the predicate factory that creates filter functions for each operator type (eq, gte, between, in, etc.).
- Implement the predicate chain with AND logic (default) and support for OR groups.
- Add predicate reordering by selectivity for performance.
- Instrument the filter engine with performance.now() timing and log execution time.

Task 3.2: Filter UI Components

Build the filter panel with the following UI components:

- RangeFilter: Dual-handle slider with numeric input fields for min/max values.
- MultiSelectFilter: Checkbox dropdown with search functionality for sectors, industries, indices.
- SingleSelectFilter: Radio button group for technical indicators (Above, Below, Neutral).
- BooleanFilter: Toggle switch for Watchlist Only, Recently Updated.
- Active filter count badge on the filter panel header.
- "Clear All Filters" button that resets all criteria.

Task 3.3: Filter Panel Layout

Design the filter panel layout:

- Collapsible sidebar on the left (320px width when expanded, icon-only when collapsed).
- Filter categories grouped under accordion sections (Fundamentals, Market Data, Classification, Technical, Custom).
- Real-time count of matching stocks displayed in the panel header: "Showing X of 5,000 stocks".
- Applied filters displayed as removable chips below the search bar.

Task 3.4: Saved Screener Presets

Implement preset filter configurations:

- "Value Stocks": P/E < 15, ROE > 15%, Debt/Equity < 0.5, Dividend Yield > 2%.
- "Growth Momentum": Revenue Growth > 20%, Profit Growth > 20%, RSI between 40-70, Price above SMA 50.
- "Large Cap Quality": Market Cap > 20,000 Cr, ROCE > 15%, Promoter Holding > 50%.
- "Technical Breakout": Price above SMA 200, RSI between 50-70, Volume above 2x average, Bollinger within bands.

Task 3.5: Filter Engine Tests

Write comprehensive tests for the filter engine:

- Test each operator type with edge cases (null values, boundary conditions).
- Test combinatorial filters (3+ criteria applied simultaneously).
- Performance test: verify < 200ms execution for 5,000 stocks with 5 simultaneous filters.
- Test preset configurations against known expected results.

Deliverable: Complete filter engine with 30+ criteria, filter panel UI, preset screeners, and 15+ filter-specific test cases.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

DAY 10: INTERACTIVE CHARTING: CANDLESTICK CHARTS WITH TECHNICAL INDICATORS

Task 4.1: Base Chart Component

Implement the StockChart component as specified in Section A3.3:

- Initialise the Lightweight Charts instance with proper configuration.
- Render candlestick data from the mock data generator (generate 252 trading days of OHLCV data per stock).
- Implement timeframe switching (1D, 1W, 1M, 3M, 1Y, 5Y) with appropriate data aggregation.
- Add crosshair with a tooltip showing OHLCV values at the cursor position.
- Responsive chart that resizes when the container dimensions change.

Task 4.2: Technical Indicator Implementation

Implement all five mandatory indicators from Section A3.2:

- SMA (20, 50, 200): Line series overlay with distinct colours and legend.
- EMA (12, 26): Line series overlay with configurable period.
- Bollinger Bands: Area series for the band with line series for upper, middle, lower.
- RSI (14): Separate pane below the main chart with overbought/oversold reference lines.
- Volume Profile: Horizontal histogram overlay on the right side of the chart.

Each indicator must have a toggle control in the chart toolbar and a settings panel for configuring parameters.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Task 4.3: Chart Interaction Features

- Zoom: Mouse wheel and pinch gesture for zooming in/out on the time axis.
- Pan: Click-and-drag to pan along the time axis.
- Reset: Double-click to reset zoom to fit all data.
- Drawing Tools (Optional/Bonus): Horizontal line, trend line, rectangle selection.
- Screenshot: Button to capture the chart as a PNG image.

Task 4.4: Chart and Grid Integration

Connect the chart to the data grid:

- Clicking a row in the data grid opens the chart for that stock.
- The chart header displays the stock symbol, company name, and current price.
- Layout: Split view with the data grid on the left (60%) and the chart on the right (40%), or full-screen chart toggle.

Task 4.5: Indicator Calculation Tests

Write unit tests for each indicator calculation function with known input/output pairs. Verify mathematical correctness against manually calculated values.

Deliverable: Interactive candlestick chart with 5 technical indicators, timeframe switching, chart-grid integration, and 10+ indicator test cases.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

DAY 11-12: WEBSOCKET SIMULATION, REAL-TIME UPDATES, AND OFFLINE SUPPORT

Task 5.1: WebSocket Server Simulation

Build the price simulation server:

- Next.js API route or standalone Node.js process using the ws library.
- Geometric Brownian motion model from Section A4.1 for realistic price movements.
- Correlated sector movements (stocks in the same sector move in the same direction with noise).
- Market hours simulation: higher volatility during open/close, lower during mid-session.
- Configurable update frequency (default: every 2 seconds per stock, randomised).
- Support for multiple simultaneous client connections.

Task 5.2: Client WebSocket Integration

Implement the client-side WebSocket hook from Section A4.2:

- Connection lifecycle management (connect, disconnect, reconnect).
- Exponential backoff reconnection (1s, 2s, 4s, 8s, 16s max).
- requestAnimationFrame-based update batching.
- Connection status indicator in the application header (green = connected, yellow = reconnecting, red = disconnected).

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Task 5.3: Real-Time Cell Updates

Implement the cell-level update mechanism:

- Flash animation on price cells when the value changes (green flash for up, red for down).
- Only re-render affected cells, not entire rows. Verify with React DevTools Profiler.
- Update the chart in real time when the selected stock receives a price update.

Task 5.4: Offline Support

Implement offline functionality:

- Cache the stock universe in IndexedDB using a service worker or idb library.
- When offline, the application should load from cache and display a banner: "Working offline - prices may be stale".
- Filter and sort functionality must work fully offline.
- When connectivity is restored, synchronise with the latest data and resume WebSocket updates.

Task 5.5: WebSocket Tests

Write tests for:

- WebSocket connection establishment and teardown.
- Reconnection behaviour with mock disconnection events.
- Batch update processing correctness.
- Offline/online transition handling.

Deliverable: Working WebSocket simulation with real-time price updates, flash animations, offline support, and 8+ WebSocket-specific test cases.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

DAY 13-14: PERFORMANCE OPTIMISATION, ACCESSIBILITY, AND BUNDLE ANALYSIS

Task 6.1: Performance Audit

Conduct a comprehensive performance audit:

- Run Lighthouse audit and document the scores for Performance, Accessibility, Best Practices, and SEO.
- Profile the application with React DevTools Profiler and identify unnecessary re-renders.
- Measure and document all performance benchmarks from Section A2.3.
- Use Chrome DevTools Performance tab to record a 30-second scroll interaction and verify > 55 FPS.
- Analyse the JavaScript bundle with next/bundle-analyzer and identify opportunities for code splitting.

Task 6.2: Performance Optimisation

Apply optimisations based on the audit results:

- Implement React.memo on all cell components with custom comparison functions.
- Add useMemo for filtered and sorted stock arrays.
- Implement dynamic imports for the charting library to reduce initial bundle size.
- Optimise images (if any) with Next.js Image component.
- Add font-display: swap for custom fonts to prevent FOIT.
- Configure caching headers for static assets.

Task 6.3: Accessibility

Ensure the application meets WCAG 2.1 AA standards:

- All interactive elements must have accessible names (aria-label or visible text).
- The data grid must have proper role attributes (grid, row, columnheader, cell).
- Colour contrast ratio must be at least 4.5:1 for all text.
- Screen reader announcements for filter changes ("Showing 342 of 5,000 stocks").
- Focus management: opening the chart should move focus to the chart area.

Task 6.4: Responsive Design

Ensure the application works on:

- Desktop: Full layout with side-by-side grid and chart (minimum 1280px width).
- Tablet: Stacked layout with collapsible filter panel (768px to 1279px).
- Mobile: Simplified single-column layout with swipe navigation between grid and chart (below 768px).

Task 6.5: Performance Report Document

Create a PERFORMANCE_REPORT.md file in the repository with:

- Lighthouse screenshots (before and after optimisation).
- Web Vitals measurements (LCP, FID, CLS, TTFB).
- Custom benchmark results (filter time, sort time, scroll FPS).
- Bundle size analysis with treemap visualisation.
- List of optimisations applied with before/after measurements.

Deliverable: Optimised application with Lighthouse score > 90 (Performance), WCAG AA compliant, responsive across breakpoints, and a comprehensive performance report.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

DAY 15: TESTING, DOCUMENTATION, DEPLOYMENT, AND GITHUB TRANSFER

Task 7.1: Test Suite Completion

Achieve minimum 70% code coverage:

- Unit tests for all utility functions, indicator calculations, and filter predicates.
- Component tests for DataGrid, FilterPanel, StockChart, and cell renderers.
- Integration tests for the complete filter-to-grid workflow.
- Run coverage report and include in the repository.

Task 7.2: README Documentation

Create a comprehensive README.md:

- Project overview and screenshots.
- Architecture diagram (component hierarchy, state flow, data flow).
- Setup instructions (step-by-step from clone to running locally).
- Environment variables documentation.
- Available scripts (dev, build, test, lint, storybook).
- Technology decisions and trade-offs.
- Known limitations and future improvements.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Task 7.3: Storybook (Optional but Recommended)

If time permits, set up Storybook with stories for:

- UI primitives (Button, Input, Select, Toggle).
- Cell renderers (PriceCell, ChangeCell, VolumeCell).
- Filter components (RangeFilter, MultiSelectFilter).
- Chart component with different indicator configurations.

Task 7.4: Deployment

Deploy the application to Vercel:

- Configure environment variables on Vercel.
- Verify the deployed application works correctly.
- Include the deployed URL in the README.

Task 7.5: Final GitHub Repository Transfer (3 hours)

CRITICAL: REPOSITORY TRANSFER PROTOCOL

This is the mandatory final submission step. Failure to complete the transfer results in project non-acceptance. No exceptions.

Pre-Transfer Checklist

- All code committed and pushed to the main branch (no dangling branches with unmerged work).
- All tests passing (run npm test and verify green output).
- Lint passing with zero warnings (run npm run lint).
- Build succeeds without errors (run npm run build).
- README is complete and accurate.
- PERFORMANCE_REPORT.md is present with all benchmarks.
- ARCHITECTURE.md is present with diagrams.
- No personal API keys, secrets, or credentials in the repository (check .env is in .gitignore).
- .gitignore properly configured (no node_modules, no .next, no .env).
- Deployed URL is accessible and functional.

Transfer Steps

- **Final Commit:** Make a final commit with the message: "Final submission - project Stock Screener v1.0 - [Your Full Name] - [YYYY-MM-DD]".
- **Tag Release:** Create a git tag: git tag -a v1.0 -m "project Stock Screener v1.0 Final Submission" && git push origin v1.0.
- **Transfer Ownership:** Navigate to Settings → Danger Zone → Transfer Ownership. Enter the organisation: ZethetaIntern. Confirm the transfer.
- **Verify Transfer:** Confirm the repository now appears under github.com/ZethetaIntern/[your-repo-name].

Deliverable: Complete test suite with 70%+ coverage, comprehensive documentation, deployed application, and successfully transferred repository under github.com/ZethetaIntern/.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART E: TOOLS AND PLATFORMS GUIDE

E1. Development Environment

Tool	Purpose	URL
Node.js 20 LTS	JavaScript runtime	https://nodejs.org/en
Visual Studio Code	Code editor (recommended)	https://code.visualstudio.com/
Git	Version control	https://git-scm.com/
GitHub	Repository hosting and submission	https://github.com/
Vercel	Deployment platform (free tier)	https://vercel.com/
Chrome DevTools	Performance profiling and debugging	Built into Google Chrome
React DevTools	React-specific debugging	Chrome Web Store extension

E2. Core Libraries (Mandatory)

Library	Version	Purpose
Next.js	14.x	React framework with App Router, SSR, API routes
React	18.x	UI component library
TypeScript	5.x	Static type checking
@tanstack/react-table	8.x	Data grid with sorting, filtering, pagination
@tanstack/react-virtual	3.x	Virtual scrolling for large datasets
lightweight-charts	4.x	Financial candlestick charting (TradingView open-source)
zustand	4.x	Lightweight state management
@tanstack/react-query	5.x	Server state management with caching

tailwindcss	3.x	Utility-first CSS framework
vitest	1.x	Unit testing framework
@testing-library/react	14.x	React component testing
ws	8.x	WebSocket server for price simulation

E3. Indian Market Data Resources

For generating realistic mock data, the intern should reference:

- NSE India: https://www.nseindia.com/ (stock listings, indices, market data)
- BSE India: https://www.bseindia.com/ (supplementary data)
- Screener.in: https://www.screener.in/ (fundamental data reference for mock generation)
- Moneycontrol: https://www.moneycontrol.com/ (sector classifications, industry data)
- Zerodha Varsity: https://zerodha.com/varsity/ (financial concepts and indicator explanations)

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART F: SUBMISSION PROTOCOL

Projects are assessed using a comprehensive 1000-point rubric covering seven dimensions: Problem Understanding, Solution Quality, Research & Analysis, Presentation & Clarity, Innovation & Creativity, Feasibility & Practicality and CV alignment. Assessment is conducted through a combination of manual review and AI-powered evaluation.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

APPENDICES

SKILLS ASSESSMENT MATRIX

Skill Domain	Specific Skills Assessed	Proficiency Levels	Evidence Required
React/Next.js	Component architecture, hooks, Server vs Client components, App Router, Suspense, Error Boundaries	Beginner / Intermediate / Advanced / Expert	Code review of component structure, hook patterns, lazy loading implementation
Data Grid Engineering	TanStack Table, virtual scrolling, column features, cell renderers, keyboard navigation	Beginner / Intermediate / Advanced / Expert	Grid performance benchmarks, scroll FPS, feature completeness
Financial Charting	Lightweight Charts, candlestick rendering, technical indicator calculation (SMA, EMA, Bollinger, RSI, Volume Profile)	Beginner / Intermediate / Advanced / Expert	Chart functionality, indicator formula correctness, real-time sync
WebSocket Integration	Connection lifecycle, exponential backoff, message batching, state synchronisation	Beginner / Intermediate / Advanced / Expert	WebSocket test suite, reconnection behaviour, latency measurements
State Management	Zustand, React Query, client/server state separation, derived state, selectors	Beginner / Intermediate / Advanced / Expert	Store architecture review, render profiling, cache strategy
Performance Engineering	Lighthouse optimisation, bundle analysis, memoisation, lazy loading, Web Vitals	Beginner / Intermediate / Advanced / Expert	Performance report with before/after metrics, profiler screenshots
Testing	Vitest, React Testing Library, unit tests, integration tests, performance benchmarks	Beginner / Intermediate / Advanced / Expert	Test coverage report, test quality review, edge case coverage

TypeScript	Strict mode, interfaces, generics, discriminated unions, type guards	Beginner / Intermediate / Advanced / Expert	Zero any usage, comprehensive type definitions, type safety in complex flows
Professional Practices	Git workflow, conventional commits, PR discipline, documentation, code review readiness	Beginner / Intermediate / Advanced / Expert	Commit history, PR descriptions, README quality, architecture docs
Financial Domain Knowledge	Equity screening concepts, fundamental ratios, technical indicators, Indian market structure	Beginner / Intermediate / Advanced / Expert	Realistic mock data, correct indicator formulas, domain terminology usage

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

GLOSSARY OF TECHNICAL TERMS

This glossary covers all technical terms used in this document.

Term	Definition
API Route (Next.js)	A server-side endpoint defined inside the app/api/ directory that handles HTTP requests. Used for mock data endpoints and WebSocket negotiation.
App Router	The Next.js 14 routing system based on the file system inside the app/ directory. Supports Server Components, Layouts, and Parallel Routes.
Bollinger Bands	A volatility indicator consisting of a middle band (SMA) and upper/lower bands at 2 standard deviations. Bands widen during volatility and narrow during consolidation.
Candlestick Chart	A financial chart showing open, high, low, and close prices for each time period. A green (hollow) body indicates close > open; a red (filled) body indicates close < open.
Client Component	A React component marked with 'use client' that executes in the browser. Required for interactivity, event handlers, hooks, and browser APIs.
CLS (Cumulative Layout Shift)	A Core Web Vital measuring visual stability. Calculated as the sum of all individual layout shift scores during the page lifecycle. Target: < 0.1.
Code Splitting	The technique of breaking JavaScript bundles into smaller chunks that load on demand, reducing initial page load time.
Compound Component	A React pattern where a parent component shares state with child components via Context, allowing flexible composition (e.g., FilterPanel with FilterGroup children).
Conventional Commits	A commit message specification: type(scope): description. Types include feat, fix, perf, test, docs, refactor, style, chore.
Custom Hook	A JavaScript function starting with 'use' that encapsulates reusable stateful logic using React hooks.
DXA (Document eXtensible Architecture)	Unit of measurement in Word documents where 1440 DXA = 1 inch. Used in this document's formatting.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

EMA (Exponential Moving Average)	A weighted moving average giving more weight to recent prices. Multiplier = 2/(period+1). More responsive than SMA to recent price changes.
Error Boundary	A React class component that catches JavaScript errors in its child tree and renders a fallback UI instead of crashing the entire application.
Exponential Backoff	A reconnection strategy where wait time increases exponentially (1s, 2s, 4s, 8s...) to avoid overwhelming a server during outages.
Geometric Brownian Motion	A mathematical model for simulating stock price movements. $dS = \mu Sdt + \sigma SdW$ where μ is drift, σ is volatility, and dW is a Wiener process increment.
Husky	A Git hooks tool that runs scripts (linting, testing) automatically before commits or pushes.
IndexedDB	A browser-based NoSQL database for storing large amounts of structured data offline. Used for caching the stock universe.
LCP (Largest Contentful Paint)	A Core Web Vital measuring loading performance. The time until the largest content element becomes visible. Target: < 2.5 seconds.
Lighthouse	An automated auditing tool by Google that measures web page quality across Performance, Accessibility, Best Practices, and SEO.
Lightweight Charts	An open-source financial charting library by TradingView. Canvas-based rendering optimised for time-series financial data.
Memoisation	An optimisation technique that caches the result of expensive function calls and returns the cached result when the same inputs occur again.
OHLCV	Open, High, Low, Close, Volume - the five standard data points for each period in a financial candlestick chart.
Overscan	In virtual scrolling, the number of extra rows rendered above and below the visible viewport to prevent blank flashes during fast scrolling.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

React Query	A server state management library (TanStack Query) that provides caching, background refetching, and stale-while-revalidate strategies for async data.
requestAnimationFrame	A browser API that schedules a callback to run before the next repaint (~60 times per second). Used for smooth animations and efficient DOM updates.
RSI (Relative Strength Index)	A momentum oscillator (0-100) measuring the speed and magnitude of price movements. RSI > 70 indicates overbought; RSI < 30 indicates oversold.
Server Component	A React component that renders on the server. Cannot use hooks, event handlers, or browser APIs. Reduces client-side JavaScript.
SMA (Simple Moving Average)	The arithmetic mean of closing prices over a specified period. SMA(20) = sum of 20 closing prices / 20.
Suspense	A React feature for declarative loading states. Wraps components that may suspend (lazy-loaded or data-fetching) and shows a fallback while loading.
TanStack Table	A headless table library providing sorting, filtering, pagination, and column management without opinionated UI components.
TanStack Virtual	A headless virtualisation library that renders only visible items plus overscan, enabling smooth scrolling through large lists.
Tree Shaking	A dead code elimination technique that removes unused exports from JavaScript bundles at build time.
TTI (Time to Interactive)	The time until the page becomes fully interactive (event handlers attached, main thread idle). Target: < 3.5 seconds.
Virtual Scrolling	Rendering only the visible rows of a large list plus a buffer, dynamically replacing DOM nodes as the user scrolls.
Volume Profile	A charting indicator showing volume distribution across price levels as horizontal bars, revealing support and resistance zones.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

WCAG 2.1 AA	Web Content Accessibility Guidelines level AA. Requires 4.5:1 contrast ratio, keyboard navigability, screen reader compatibility.
Web Worker	A browser API for running scripts in background threads without blocking the main UI thread. Useful for heavy computations.
WebSocket	A protocol providing full-duplex communication over a single TCP connection. Used for real-time data streaming.
Zustand	A lightweight state management library for React. Uses hooks-based API with selector subscriptions for granular re-renders.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

TECHNICAL INDICATOR FORMULA REFERENCE

This appendix provides the complete mathematical specification for each mandatory indicator. The intern's implementation must produce outputs that match these formulas within a tolerance of 0.01.

B1. Simple Moving Average (SMA)

Formula: $SMA(n, t) = (1/n) * \Sigma(i=0 \text{ to } n-1) \text{ of } Close(t-i)$

Where n is the period length and t is the current bar index. The SMA is undefined for bars where $t < n-1$.

Verification Data: Given closing prices [100, 102, 104, 103, 105, 107, 106, 108, 110, 109], the SMA(5) values starting from index 4 should be [102.8, 104.2, 105.0, 105.8, 107.2, 108.0].

B2. Exponential Moving Average (EMA)

Multiplier: $k = 2 / (n + 1)$

Initial EMA: $EMA(n, n-1) = SMA(n, n-1)$ (use SMA for the first value)

Recursive: $EMA(n, t) = Close(t) * k + EMA(n, t-1) * (1 - k)$

Verification Data: Given the same closing prices above, EMA(5) with $k = 0.3333$: EMA at index 4 = 102.8 (same as SMA). EMA at index 5 = 107 * 0.3333 + 102.8 * 0.6667 = 104.20. The intern must verify their implementation against these values.

B3. Bollinger Bands

Middle Band: $MB(t) = SMA(20, t)$

Standard Deviation: $\sigma(t) = sqrt((1/20) * \Sigma(i=0 \text{ to } 19) \text{ of } (Close(t-i) - SMA(20, t))^2)$

Upper Band: $UB(t) = MB(t) + 2 * \sigma(t)$

Lower Band: $LB(t) = MB(t) - 2 * \sigma(t)$

Implementation Note: Use the population standard deviation (divide by n), not the sample standard deviation (divide by n-1). Financial convention uses population standard deviation for Bollinger Bands.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

B4. Relative Strength Index (RSI)

Step 1: Calculate price changes: Δ(t) = Close(t) - Close(t-1)

Step 2: Separate gains and losses: Gain(t) = max(Δ(t), 0), Loss(t) = |min(Δ(t), 0)|

Step 3: First average (SMA method): AvgGain(14) = mean of first 14 gains, AvgLoss(14) = mean of first 14 losses

Step 4: Subsequent averages (smoothed): AvgGain(t) = (AvgGain(t-1) * 13 + Gain(t)) / 14

Step 5: RS = AvgGain / AvgLoss

Step 6: RSI = 100 - (100 / (1 + RS))

Edge Cases: If AvgLoss = 0, RSI = 100. If AvgGain = 0, RSI = 0. The intern must handle these edge cases explicitly.

B5. Volume Profile

Step 1: Determine the visible price range: PriceMin = min(Low) over visible period, PriceMax = max(High) over visible period.

Step 2: Divide the price range into N buckets (recommended: 30-50 buckets): BucketSize = (PriceMax - PriceMin) / N.

Step 3: For each candle, distribute the candle's volume proportionally across the price buckets that the candle spans (from Low to High).

Step 4: Render each bucket as a horizontal bar, with bar length proportional to the accumulated volume in that bucket.

Implementation Note: The Volume Profile must update when the user zooms or pans the chart, as the visible price range changes. This requires recalculation on every visible range change.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

INDIAN EQUITY MARKET CONTEXT

This appendix provides essential context about the Indian equity market that the intern must understand to create realistic mock data and appropriate filter criteria.

C1. Market Structure

- **NSE (National Stock Exchange):** India's largest stock exchange by trading volume. Over 2,000 listed companies. Benchmark index: NIFTY 50.
- **BSE (Bombay Stock Exchange):** Asia's oldest stock exchange. Over 5,000 listed companies. Benchmark index: BSE Sensex.
- **Trading Hours:** Pre-open session: 9:00 AM to 9:15 AM IST. Normal trading: 9:15 AM to 3:30 PM IST. Post-close session: 3:40 PM to 4:00 PM IST.
- **Currency:** Indian Rupee (INR). All prices and market cap values in the screener must be displayed in INR.
- **Lot Size:** Equity delivery has no minimum lot size (1 share minimum). Futures and options have predefined lot sizes.

C2. Market Capitalisation Categories (SEBI Definition)

Category	SEBI Definition	Approximate Range (INR Cr)	Stocks in Universe
Large Cap	Rank 1-100 by full market capitalisation	50,000+	~100
Mid Cap	Rank 101-250 by full market capitalisation	10,000 - 49,999	~150
Small Cap	Rank 251 and below by full market capitalisation	Below 10,000	~4,750

C3. Key Financial Ratios in Indian Context

Indian market conventions differ slightly from global standards. The intern should note:

- **P/E Ratio:** The NIFTY 50 historical P/E ranges from 12 (crisis levels) to 30 (euphoria). A P/E of 20-25 is considered fair value for the broad market.
- **Promoter Holding:** Unique to Indian markets. SEBI requires promoters to hold a minimum of 20% in listed companies. Promoter holding above 60% is generally considered a positive signal of skin in the game.

- **Dividend Yield:** Indian companies typically have lower dividend yields (1-3%) compared to US companies. PSU (Public Sector Undertaking) companies and old-economy stocks tend to have higher yields.
- **ROE and ROCE:** ROE above 15% is considered good. ROCE above 20% is excellent. These are the most commonly used profitability metrics in Indian equity analysis.
- **Debt-to-Equity:** Banking and NBFC stocks have naturally high D/E ratios (5-15x). For non-financial companies, D/E below 0.5 is considered conservative, and above 2.0 is considered aggressive.

C4. Number Formatting Conventions

Indian number formatting differs from Western conventions. The screener must use Indian number system formatting:

Value (INR)	Western Format	Indian Format	Abbreviation
100,000	100,000	1,00,000	1 Lakh (1L)
10,000,000	10,000,000	1,00,00,000	1 Crore (1Cr)
1,000,000,000	1,000,000,000	1,00,00,00,000	100 Crore (100Cr)
100,000,000,000	100,000,000,000	1,00,00,00,00,000	1 Lakh Crore (1LCr)

Implementation Requirement: All market cap values must be displayed in Crore (Cr) with Indian comma formatting. Volume must be displayed with appropriate abbreviations (L for Lakh, Cr for Crore, K for Thousand). Prices must use two decimal places with INR symbol (₹).

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

END OF PROJECT DOCUMENT